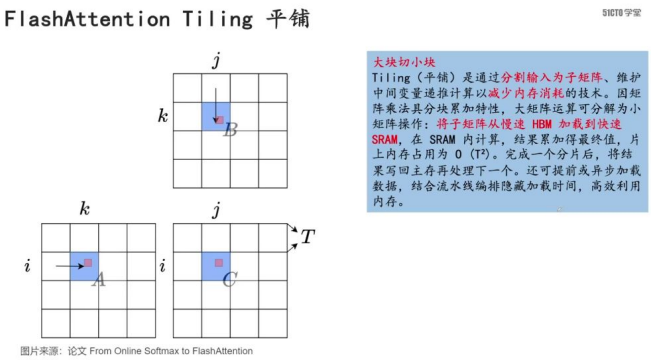
以下为AI生成的图文笔记的内容

一、FlashAttention 00:00

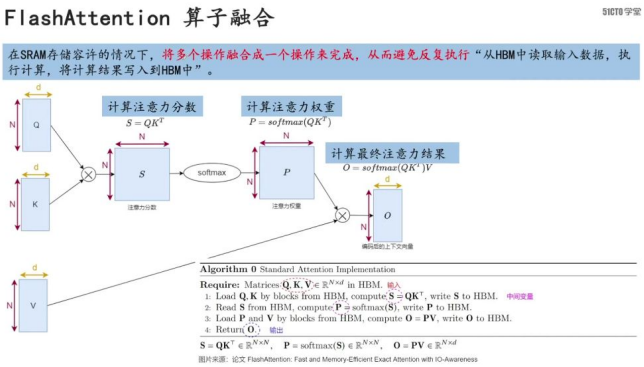
1. Tilling平铺 00:14



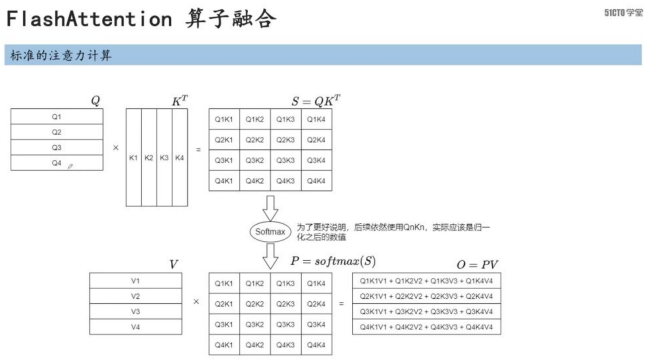
-
- **技术原理**：将大矩阵分割为子矩阵进行分块计算，通过维护中间变量递推减少内存消耗。利用矩阵乘法的分块累加特性，将子矩阵从HBM显存加载到SRAM快速内存计算。
- **实现方式**：每次处理一个分片后写回主存，通过提前/异步加载数据结合流水线编排隐藏加载时间，片上内存占用为$O(T^2)$。
- **优势**：避免一次性加载大矩阵导致内存溢出，特别适合处理长序列（如100个token以上）的注意力计算场景。

2. 算子融合 01:04

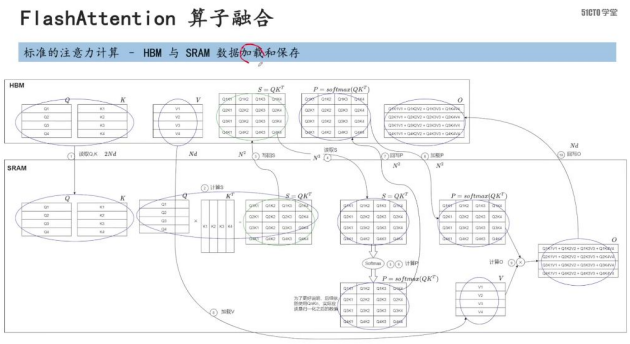
1）标准的注意力计算 02:31



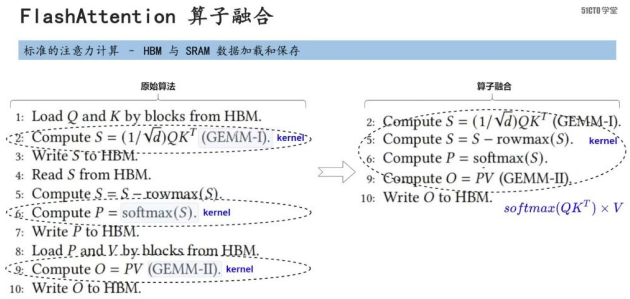
-
- **计算步骤**：
  - 从HBM加载Q,K，计算$S = QK^T$后写回HBM
  - 读取S计算$P = softmax(S)$后写回HBM
  - 加载P,V计算$O = PV$后写回HBM
- **内存瓶颈**：中间变量$S \in R^{N \times N}$和$P \in R^{N \times N}$需要反复读写HBM，当N增大时（如4个token扩展为100个），显存带宽成为性能瓶颈。



-
- **具体案例**：以4个token为例，QK^T产生16个中间值（如$Q1K1…Q4K4$），经过softmax后与V进行加权求和，整个过程需要多次显存交互。
- HBM与SRAM数据加载和保存 03:35

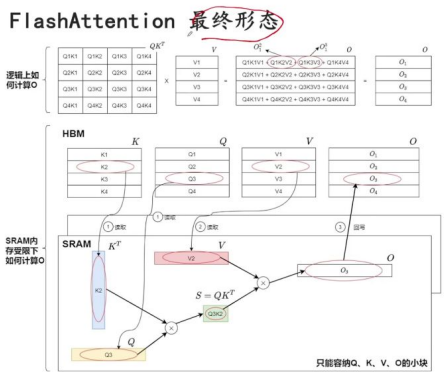


  -
  - **数据流问题**：传统方法需要8次显存交互（读取QK→计算S→写回S→读取S→计算P→写回P→读取PV→写回O），形成"显存墙"效应。
  - **关键矛盾**：SRAM速度快但容量小（约KB级），HBM容量大但带宽有限，频繁数据搬运导致计算效率低下。
- 原始算法算子融合 05:07



  -
  - **融合方案**：将QK^T计算、rowmax、softmax、PV计算四个核心操作合并为单个内核计算
  - **技术突破**：
    - 中间结果保留在SRAM寄存器/缓存中
    - 仅最终输出O需要写回HBM
    - 显存访问次数从$O(N^2)$降至O(N)

2）最终态 06:32



- **执行逻辑**:
  - 并行加载Q,K,V的小分块到SRAM
  - 在SRAM内完成$S = QK^T \rightarrow P = softmax(S) \rightarrow O = PV$全流程
  - 分块累加最终结果写回HBM
- **核心优势**:
  - 类似"自助餐"式处理：小分块多次加载计算
  - SRAM内完成90%计算，HBM仅存储初始输入和最终输出
  - 理论显存访问量减少约5-20倍（具体取决于序列长度）

二、知识小结

知识点	核心内容	技术难点/关键点	实现方法
矩阵切分（Tiling）	将大矩阵拆解为小块子矩阵，减少单次内存消耗	**显存与SRAM的速度差异**	分块加载至GPU内核，运算后暂存SRAM
算子融合	合并多个计算步骤（如QK^T、softmax、PV）避免显存反复读写	**中间结果存储优化**	在SRAM内完成全部计算，仅最终结果写回显存
传统注意力机制缺陷	需多次显存读写（QK^T→写回→softmax→写回→PV→写回）	**高I/O延迟**	通过**切块+融合**减少显存访问次数
Flash Attention优化原理	1. **切块并行处理** 2. **SRAM缓存中间值** 3. **流水线编排**	**SRAM容量限制**	动态加载小块数据，类似"自助餐"式分批处理
显存与SRAM协作	HBM（慢速显存）→ SRAM（高速缓存）→ 寄存器计算	**数据搬运效率**	仅最终输出写回HBM，避免中间步骤I/O瓶颈